Claude Code ワークフロー

コーディングエージェント入門

伊藤 恒平

2026

コーディングエージェントとは

定義

AI がターミナル上でコードを読み書きし、ビルド・テスト・Git 操作まで**自律的に実行**するツール

	チャット AI	コーディングエージェント
動作	対話（コピペが必要）	ファイルを直接編集
範囲	1 ファイルの断片	プロジェクト全体
実行	ユーザーが手動実行	ビルド・テストも自律実行
Git	関与しない	コミット・PR 作成まで

なぜ CLI なのか: IDE プラグインと違い、SSH 先・Docker 内・CI 上など**どこでも動く**

Claude Code とは

概要

Anthropic 公式の CLI エージェント。ターミナルから Claude を呼び出し、コード読み書き・ビルド・テスト・Git 操作を自律的に実行する

機能	説明
CLAUDE.md	プロジェクト固有の指示書（常に読み込まれる）
Memory	セッション間で記憶を永続化（4 タイプ）
Plan Mode	Shift+Tab で読み取り専用の設計モード
Subagent	並列タスク・重い処理を子プロセスに委任
Hooks & Skills	自動化とカスタムワークフロー
Git 統合	コミット・PR 作成・ブランチ操作

インストール: `npm install -g @anthropic-ai/claude-code`　起動: `cd your-project && claude`

なぜ「計画」が必要か

AI に自由にコードを書かせると何が起きるか

- 意図と違う設計で大量のファイルを変更 → **手戻りコスト大**
- テストなしで実装を進める → バグの発見が遅れる
- 既存コードの意図を誤解 → 動いていた機能が壊れる

解決策:「承認」ステップを挟む

1. AI に計画を出させる（コードは書かせない）
2. 人間が計画をレビューして承認/修正
3. 承認後に初めて実装を開始

原則:「考える」と「書く」を分離する。これが手戻りを防ぐ最大のコツ

Plan Mode — 設計してから書く

Plan Mode とは

Shift+Tab でトグル。コード編集を**できない**読み取り専用モード。
調査・設計に集中させ、方針が固まってから実装に移る

有効な場面

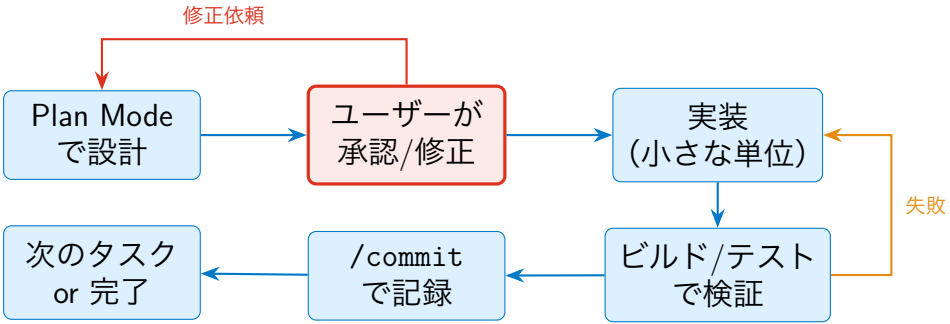
- 複数ファイルにまたがる変更
- アーキテクチャの判断が必要
- 失敗コストが高い作業
- 初めて触るコードベース

Plan Mode でできること

ファイル読み取り・検索
コードベースの構造分析
変更計画の策定・提案
影響範囲の調査

× ファイル編集・作成
× コマンド実行

Plan → Approve → Implement サイクル



赤枠 = 人間の承認ポイント。ここで方向修正できるから安全に AI を使える

Plan Mode の実践例

例:「CSV ファイルを読み込んでグラフを表示する機能を追加して」

Plan Mode で Claude が出す計画の例:

```
## Plan: CSV 読み込み + グラフ表示機能

### 変更対象ファイル
1. src/csv_reader.py   (新規) - CSV 読み込み処理
2. src/plot.py         (新規) - グラフ描画処理
3. src/main.py         (既存) - メニューに項目追加
4. tests/test_csv.py   (新規) - 読み込みのテスト

### 設計判断
- CSV ライブラリ: 標準の csv モジュールを使用
- グラフ描画: matplotlib を使用
- ファイル選択: コマンドライン引数で指定

### 影響範囲
- 既存の機能には影響なし
- requirements.txt に matplotlib を追加
```

ユーザーが「3 ファイルに分けず 1 ファイルにまとめて」→ 計画を修正 → 承認 → 実装開始

なぜ Git が重要か

AI がファイルを自律的に変更する世界では

変更履歴がなければ「何が変わったか」「なぜ変わったか」が追えない。

Git なしで AI にコードを書かせるのは、セーブなしでゲームをするようなもの

Git が提供する安全網

- 変更の差分を確認できる
- いつでも前の状態に戻せる
- 誰が（AI/人間）何を変えたか記録
- ブランチで安全に実験できる

Claude Code の Git 統合

- 変更後に自動コミット提案
- Conventional Commits 準拠
- PR 作成・差分要約
- コミット履歴から文脈を読む

原則: AI と協働するなら Git は必須。「こまめにコミット」が最大の保険

コミット戦略

Conventional Commits + Next steps

```
feat(csv): add CSV import and graph display

Changes:
- CSV 読み込み処理を実装
- matplotlib でグラフ描画機能を追加
- メインメニューに項目を追加

Next steps:
- tests/test_csv.py でテストを実行
- 大きな CSV ファイルでの動作確認
```

コミットのタイミング:

- 1 機能 = 1 コミット（小さな単位）
- テスト/ビルド成功後
- 作業の区切り・中断時

Next steps の効果:

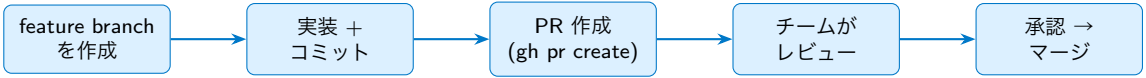
- 次のセッション開始時の起点になる
- 具体的なコマンド・検証項目を含む
- AI が「前回の続き」を即座に把握

GitHub 連携 — PR とレビュー

Claude Code で PR を作成する

「この変更で `PR` を作成して」と指示するだけで:

1. 変更差分を分析して PR タイトル・本文を自動生成
2. `gh pr create` で GitHub に PR を作成
3. レビューコメントへの対応も指示可能



Tips: Claude Code は PR のレビューコメントを読んで修正コミットも作れる

ブランチ戦略と安全運用

main ブランチを直接変更しない

AI が `git push origin main` を実行すると、レビューなしで本番に反映される。
必ず feature branch → PR → merge のフローを守る

推奨フロー

1. `git checkout -b feat/auth`
2. 実装・テスト・コミット
3. `gh pr create` で PR 作成
4. レビュー・承認後にマージ
5. ブランチを削除

権限制限で安全に

`settings.json` で制御:

- `permissions.deny` で `git push origin main` をブロック
- `--allowedTools` で使えるツールを制限
- GitHub の branch protection ルールと組み合わせる

原則: AI に「何をさせるか」だけでなく「何をさせないか」も設定する

CLAUDE.md — プロジェクトの知識を蓄積する

階層構造（すべて同時に読み込まれる）

`~/.claude/CLAUDE.md`	グローバル設定（全プロジェクト共通）
`<repo>/CLAUDE.md`	プロジェクト設定（Git 管理）
`<subdir>/CLAUDE.md`	ディレクトリ固有の設定
`.claude/rules/*.md`	パス限定ルール（paths フロントマター）

書くべき内容

- ビルド・テストコマンド / コーディング規約 / コミット形式
- **やってはいけないこと**（アンチパターン）← 最も効果が高い
- プロジェクト構造の概要

Memory — AI が学習する仕組み

4 つのメモリタイプ

Type	用途
user	ユーザーの専門性・好み
feedback	修正指示・NG パターン
project	進行中の作業・期限
reference	外部リソースへのリンク

feedback メモリの例

テストは必ず実行してから
コミットすること。
→ 全セッションで自動遵守

SQL は ORM 経由で書くこと。
生 SQL は禁止。

→ 次回から自動的に従う

最も強力なのは feedback メモリ。一度「こうしないで」と伝えれば、**全ての将来のセッション**で Claude が従う。

Claude Code にとっての「知識」とは

Claude Code は毎回まっさらな状態で起動する

前回の会話は覚えていない。ではどうやって「このプロジェクトのルール」を知るのか？

→ **ファイルに書かれた情報を毎回読み込む**ことで知識を得る

知識の種類	具体例
プロジェクトのルール	「テストは pytest で実行」「コミットは英語で書く」
過去の失敗から学んだこと	「この関数は変更禁止」「SQL は直接書かない」
ユーザーの好み	「説明は日本語で」「コメントは簡潔に」
カスタムコマンド	`/commit` で自動コミット等

これらの知識は**すべてファイルとして保存**されている。

持ち運べる知識 vs この PC だけの知識

持ち運べる知識（Git 管理）

プロジェクトのフォルダ内に保存。

`git clone` すれば別 PC でも復元

ファイル	役割
`CLAUDE.md`	プロジェクトのルールブック
`.claude/skills/`	カスタムコマンド
`.claude/settings.json`	権限・Hook の設定

この PC だけの知識（ローカル）

あなたの PC の中だけに保存。

別の PC には**引き継がれない**

ファイル	役割
Memory	過去の会話で学んだこと
グローバル CLAUDE.md	全プロジェクト共通の個人設定

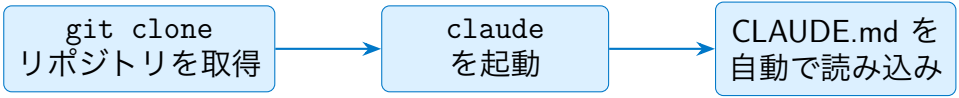
重要: Memory に保存した大事なルールは、`CLAUDE.md` にも書いておく。

そうすれば「持ち運べる知識」になり、別 PC・別メンバーにも引き継がれる

git clone すれば AI の知識も一緒に来る

別の PC やチームメンバーへの共有

CLAUDE.md や Skills は Git 管理。クローンすれば **AI の知識も自動的に手に入る**

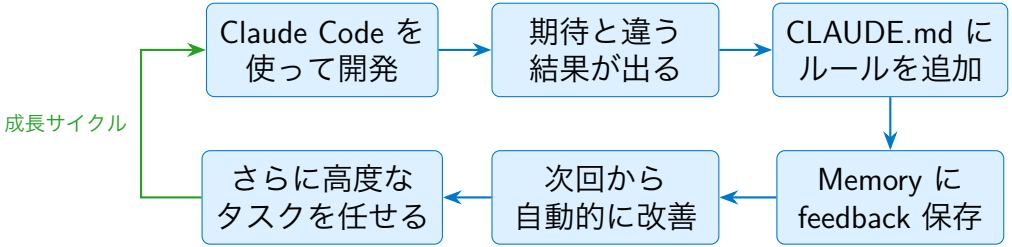


→ 初日から AI がルールを理解

GitHub = AI の知識のハブ

- 新メンバーが `git clone` → 同じルールを共有、教育コスト削減
- CLAUDE.md の改善を PR で提案 → チームで AI ルールをレビュー
- コミットの Next steps → 別の人が作業を引き継ぎやすい

Claude Code を育てる



1 ヶ月目: ビルドコマンドと基本規約だけの CLAUDE.md

3 ヶ月目: 失敗事例から構築されたチェックリスト・禁止事項の集大成

投資対効果: ドキュメント化の時間は「AI への教育投資」。使い込むほど Claude は賢くなる

Hooks とは — イベント駆動の自動化

概念

Claude がツールを使う前後に、シェルコマンドを**自動実行**する仕組み。
「ファイルを保存したら自動で lint を走らせる」のようなことができる

イベント	用途
PreToolUse	ツール実行前。危険な操作をブロック
PostToolUse	ツール実行後。自動 lint / format
Notification	通知発生時。Slack / メール転送
Stop	応答完了後。自動テスト実行

設定場所: `.claude/settings.json`（Git 管理）または `settings.local.json`（ローカル専用）

prompt-based（テキスト判定）/ agent-based（AI 判定）も選択可能

Hooks の作り方と活用例

例 1: 自動フォーマット

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "command": "npx prettier
        --write $TOOL_INPUT_PATH"
    }]
  }
}
```

例 2: 危険コマンドブロック

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "command": "if echo
        $TOOL_INPUT | grep -q
        'rm -rf'; then
        exit 2; fi"
    }]
  }
}
```

その他の活用例

Notification Hook: 長時間タスク完了時にデスクトップ通知を送る

Stop Hook: Claude の応答完了後に自動でテストスイートを実行

Skills とは — カスタムコマンド

概念

複雑なワークフローを `/commit` のような 1 コマンドで実行する仕組み。

`.claude/skills/` に `SKILL.md` で定義する

SKILL.md の構造

```
---
name: commit
description: ガイドライン準拠
  のGitコミットを作成
allowed-tools:
  - Bash
  - Read
---
1. git diff で変更を確認
2. 規約に従いメッセージ作成
3. git commit を実行
引数: $ARGUMENTS
```

組み込み Skills

`/commit` — ガイドライン準拠の Git コミットを自動作成

`/simplify` — コード品質チェック

`/batch` — 並列ワークツリーで大規模変更を一括処理

Tips: 繰り返す作業を Skill に

Subagent — 並列処理とコンテキスト保護

タイプ	用途	説明
general-purpose	汎用タスク	全ツール利用可。複雑な作業を委任
Explore	コード検索	読み取り専用。高速なコードベース探索
Plan	設計	読み取り専用。アーキテクチャ設計

活用例

- **テスト並列実行** — 複数のテストスイートを同時に走らせる
- **コードベース調査** — 大規模リポジトリの構造を探索させる
- **バックグラウンドビルド** — ビルドを裏で実行し完了通知を受け取る

コンテキスト保護: 大量の検索結果やログをメインに持ち込まず、サブエージェントに要約させる

コンテキスト管理

コンテキストウィンドウは最重要リソース

Claude が一度に扱える情報量には上限。無駄に消費すると性能が劣化し指示を忘れる

手法	説明
`/compact`	会話履歴を要約して圧縮（~95% で自動発動）
Subagent 委任	大量の検索結果をメインに持ち込まない
Memory 永続化	繰り返し伝える情報はメモリに保存
CLAUDE.md 簡潔化	毎回読み込まれるので 200 行以内に

Tips: セッションが長くなったら `/compact` で圧縮。重要な情報は Memory に保存しておけば圧縮後も失われない

初心者がやりがちな失敗

失敗パターン	何が起きるか
Plan なしでいきなり実装させる	意図と違う設計で大量変更 → やり直し
Git を使わない	AI の変更を追えない → 戻せない
CLAUDE.md を書かない	毎回同じ指示を繰り返す → 非効率
main に直接 push	レビューなしで本番反映 → 障害リスク
Hooks を使わず手作業を繰り返す	lint 忘れ・フォーマット崩れが頻発
Memory を活用しない	同じミスを何度も指摘する必要がある

共通の原因: AI を「魔法の箱」として扱ってしまう →**対策:** ガードレール（Plan, Git, CLAUDE.md）を先に整える

Top 10 Tips

1. **Plan Mode で考えてから書く** — 5 分の計画で 1 時間の手戻りを防ぐ
2. **Git は必須** — こまめにコミット、Next steps を含める
3. **feature branch → PR → merge** — main を直接触らない
4. **CLAUDE.md は AI への教育投資** — 書くほど賢くなる
5. **ガードレール > 指示** —「やるな」の方が「やれ」より効果的
6. **feedback メモリで学習** — 同じミスを二度と繰り返さない
7. **Claude Code を育てる** — 失敗するたびにルールを追加
8. **Hooks で自動化** — 手作業の繰り返しを排除
9. **Skills でワークフロー化** — 複雑な手順を 1 コマンドに
10. **GitHub = AI の知識のハブ** — clone すれば知識も一緒に来る

参考資料 / はじめよう

リソース	URL
公式ドキュメント	https://docs.anthropic.com/en/docs/claude-code
ベストプラクティス	https://docs.anthropic.com/en/docs/claude-code/best-practices
Claude Code GitHub	https://github.com/anthropics/claude-code

Claude Code を始めるには

```
npm install -g @anthropic-ai/claude-code
cd your-project && claude
```